

SDD Playbook — Guía de bolsillo

Toolkit: `sdd-toolkit` · **Basado en:** WORKFLOW v11 · **Autor:** Martin Bortagaray **Propósito:** Guía paso a paso imprimible de cada flujo del proceso SDD con los comandos del plugin de Claude Code. Cada sección es autocontenida (pensada para imprimirse como ficha).

Nota de invocación: todos los comandos se invocan con el prefijo del plugin: `/sdd-toolkit:<comando>` (ej. `/sdd-toolkit:sdd-discovery`). Por brevedad, en este documento los escribo sin el prefijo. Tipeá `/sdd` en Claude Code para autocompletarlos.

0 · Referencia rápida

Los 9 comandos

Comando	Fase	Qué hace
sdd-discovery	2	Te interroga para extraer decisiones. No decide por vos.
sdd-spec	3	Redacta el draft de una spec nueva .
sdd-modify-spec	3 (sobre feature existente)	Edita una spec existente (Implemented/As-built).
sdd-adversarial-spec	4	Ataca la spec buscando gaps (subagente, contexto limpio).
sdd-verify	5→6	Semáforo VERDE/AMARILLO/ROJO antes de codear.
sdd-codegen	6	Genera código, una capa por vez .
sdd-adversarial-code	6	Ataca el código contra la spec (subagente).
sdd-bugfix	§9	Trata un bug con trazabilidad (clasifica A/B/C).
sdd-commit	—	Commit inteligente con convención SDD.

Las 5 reglas no negociables

1. La IA **nunca toma decisiones de producto** sin preguntar. Aunque parezca obvio.
2. La IA **marca cada decisión por defecto** en una sección clara, para validarla.
3. **Leo la spec entera** antes de aprobarla. No "una mirada".
4. **Toda spec pasa por pasada adversaria** antes de aprobarse. No se saltea.
5. Si una spec aprobada estaba mal: **subo versión y documento**. No corrijo en silencio.

Jerarquía de gobierno (la capa superior gobierna)

Setup foundational → Feature Spec → Código

El código nunca contradice la spec; la spec nunca contradice el setup foundational.

Árbol de decisión — ¿qué tipo de cambio tengo?

```
¿El sistema NO hace lo que la spec dice?
└─ Sí → BUG → sdd-bugfix    (Tipo A: spec ok / Tipo B: spec mal)

¿Hace lo correcto, pero quiero agregar/cambiar funcionalidad?
├─ ¿Parte de una feature existente? → sdd-modify-spec ← caso dominante en gitelli
└─ ¿Capacidad nueva e independiente? → sdd-spec        (spec nueva)

¿El cambio se repite en varias features o es arquitectónico?
└─ Primero el setup foundational (sdd/foundation/), después la spec
```

Flujo A · Feature nueva (ciclo completo)

Capacidad funcional nueva e independiente. Recorre las 6 fases.

Fase 1 — Definir necesidad

- ☐ Escribo el problema en 1-2 líneas + notas crudas (restricciones, dependencias). Sin formato.

Fase 2 — Discovery → `sdd-discovery "necesidad"`

- ☐ Respondo las preguntas: decisión concreta / "no decidido aún" / "ayúdame a pensarlo".
- ☐ No esquivo decisiones ("es estándar", "lo vemos después" → el comando me frena).
- ☐ Output: documento de respuestas + pendientes + dependencias.

Antes de Fase 3: resuelvo todos los [PENDIENTE] (consulta a agrónomo/socio/experto si hace falta) y valido cada decisión por defecto.

Fase 3 — Draft de spec → `sdd-spec [discovery] [ID]`

- ☐ La IA redacta llenando el template. No "mejora" mis decisiones, las traduce.
- ☐ Si aparece un gap de producto → me frena. Lo resuelvo.
- ☐ Output: `sdd/specs/<ID>.md` en estado **Draft**. Actualizo INDEX.

Fase 4 — Pasada adversaria → `sdd-adversarial-spec [ID] 1`

- ☐ Corre en subagente (contexto limpio). Devuelve hallazgos por 10 categorías.
- ☐ Clasifico: sólidos para iterar / zona gris / descartables (estilo, casos teóricos).
- ☐ Proceso los aceptados, documento en sub-sección 10.2, subo versión. Estado → **Review**.
- ☐ **Máximo 2 pasadas.** Si la pasada 2 sigue mostrando problemas de fondo: parar y repensar.

Fase 5 — Aprobación (lectura crítica con cabeza fresca)

- ☐ Pre-requisitos: cero [PENDIENTE] / TBD, dudas externas resueltas y escritas, ≥1 pasada adversaria hecha.
- ☐ **Señales de aprobar con criterio:** puedo nombrar las 3 decisiones clave, identifico un trade-off, sé cuál es la parte más débil y por qué la acepto.
- ☐ **Señales de aprobar por inercia (PARAR):** "se ve bien" sin más, no recuerdo qué se decidió, estoy cansado/apurado.
- ☐ Si OK → estado **Approved**, registro fecha + versión.

Fase 5→6 — Verificación → `sdd-verify [ID]`

- ☐ ● VERDE → sigo. ● AMARILLO → decido conscientemente y registro. ● ROJO → resuelvo bloqueantes.

Fase 6 — Codegen por capas → `sdd-codegen [ID] [capa]` (capas 1→2→3→4)

Por cada capa:

- ☐ La IA genera **primero la tabla comparativa** (spec vs. implementación). La reviso.
- ☐ Confirmo "tabla ok, generará". Recién ahí escribe código + tests.
- ☐ Reviso decisiones técnicas por defecto. Sin sobre-ingeniería.
- ☐ `sdd-adversarial-code [ID] [capa]` (subagente) → proceso bloqueantes ya, no-bloqueantes a `DEBT.md`.
- ☐ Tests de la capa pasan. Recién entonces paso a la capa siguiente.
- ☐ Capa 2+: cargar schema real de la DB. Capa 1: migraciones **append-only**.

Cierre → `sdd-commit`

- ☐ **Un solo commit por feature completa y verificada** (no por capa). Ver Flujo D para deploy.

Flujo B · Modificar feature existente (spec Implemented/As-built)

*Caso dominante en gitelli. Cambio de producto sobre una feature que ya existe. Re-entra al ciclo **acotado al delta**.*

Paso 0 — Clasifico (ver árbol de decisión en §0)

- ☐ ¿Es bug? → Flujo C. ¿Capacidad nueva independiente? → Flujo A. ¿Se repite en varias features? → primero el foundation.
- ☐ Si es agregar/cambiar funcionalidad de esta feature → sigo acá.

Paso 1 — Leo la spec existente con ojo crítico (Regla 3)

- ☐ ¿Refleja bien el comportamiento actual? **Ojo con las as-built:** suelen tener huecos.
- ☐ Si la base está floja, completarla es parte de este cambio (espíritu de Regla 5).

Paso 2 — Discovery del delta → `sdd-discovery` "qué agrego/cambio"

- ☐ Acotado al cambio, no re-discovery de toda la feature. Resuelvo pendientes y valido decisiones por defecto.

Paso 3 — Modifico la spec → `sdd-modify-spec` [ID] [discovery]

- ☐ Edita **solo las secciones afectadas** (no reescribe). Sube versión + entrada de changelog.
- ☐ Si toca el modelo de datos → en código será una **nueva migración append-only**.
- ☐ Si el delta es de producto no resuelto, o choca con el foundation → me frena.

Paso 4 — Adversaria + verificación → `sdd-adversarial-spec` → (apruebo) → `sdd-verify`

- ☐ La Regla 4 **no se relaja** por ser "solo un cambio".

Paso 5 — Codegen del delta → `sdd-codegen` (solo las capas que el cambio toca)

- ☐ Si el delta no afecta el modelo de datos, no se regenera Capa 1.
 - ☐ `sdd-adversarial-code` por capa tocada → `sdd-commit`.
-

Flujo C · Bug

Un bug no rompe "spec antes que código". Cambia la velocidad, no la trazabilidad.

Paso 1 — Clasifico ANTES de tocar código → `sdd-bugfix "descripción"`

Tipo	Definición	Qué corrijo
A — Implementación	El código no cumple lo que la spec dice. La spec estaba bien.	Solo el código.
B — Spec	La spec no contempló el caso / era ambigua. El código hizo lo que decía.	Primero la spec (nueva versión), luego el código.
C — Negocio	El sistema cumple la spec, pero el negocio cambió de opinión.	No es bug → es feature/modificación (Flujo A o B).

Si dudo entre A y B, casi siempre es **B**.

Paso 2 — Artefacto `sdd/bugs/bugfix-NNN.md` (estado **Abierto**)

- ☐ Descripción (observado vs esperado), reproducción, root cause, criterio de aceptación testeable, severidad, tipo.

Paso 3 — Según severidad

- ☐ **Crítico** (prod caída / dato corrupto): fix mínimo ya, artefacto Abierto el mismo día, completar en 24h.
- ☐ **Alto/Medio/Bajo**: proceso completo sin atajos. Si **Tipo B** → **actualizo la spec primero** (sube versión).

Paso 4 — Test de regresión (regla de cierre)

- ☐ Escribo el test **antes** del fix: debe **fallar** con el código actual.
- ☐ Genero el fix **mínimo** (sin refactor, sin mejoras de paso).
- ☐ El test **pasa** y queda en la suite. Lo referencio en el `bugfix-NNN.md`.

Paso 5 — Cierre

- ☐ Estado **Cerrado** solo con: fix verificado + test en verde + spec actualizada si era Tipo B. Commit con `sdd-commit` (prefijo `fix(bugfix-NNN):`).

Flujo D · Commit y deploy (ramas)

Modelo GitHub Flow simplificado. **Nadie** *pushea* **directo** a *main* , **ni** *yo*.

```
main      → producción. Siempre estable.
staging    → validación del cliente.
feature/*  → una rama por spec (feature/SPEC-ID).
hotfix/*   → fixes urgentes de producción.
```

Durante la feature

- ☐ Al iniciar Fase 6: `git checkout main && git pull` → `git checkout -b feature/SPEC-ID` .
- ☐ **Un solo commit** cuando la feature está completa y verificada (no por capa). Formato: `feat(SPEC-ID): descripción` .

Deploy

- ☐ A staging: `git checkout staging && git pull && git merge feature/SPEC-ID && git push` → deploy manual → cliente valida.
- ☐ Cliente aprueba → a producción: `git checkout main && git pull && git merge feature/SPEC-ID && git push` → deploy manual.
- ☐ `git branch -d feature/SPEC-ID` . Actualizo INDEX: spec **Approved** → **Implemented**.

Hotfix

- ☐ `git checkout -b hotfix/desc` desde main → fix → merge a main **y** a staging → borrar rama.
-

Flujo E · Instalar / actualizar el plugin

Se hace **una vez por máquina** (scope user → todos los proyectos). Requiere el CLI `cLaude` .

Instalar (terminal / PowerShell)

```
claude plugin marketplace add "C:\Users\mrtbr\CODE\sdd-toolkit" # local (dev)
# o desde GitHub: claude plugin marketplace add martin-bortagaray/sdd-toolkit
claude plugin install sdd-toolkit@sdd-toolkit # --scope user por default
claude plugin list # verificar
```

- ☐ **Reiniciar** Claude Code para que carguen los comandos. Verificar con `/sdd` .

Actualizar (después de editar el toolkit)

```
claude plugin marketplace update sdd-toolkit
```

- ☐ Si instalaste desde GitHub: `git commit && git push` **antes** del update. Reiniciar la sesión.
-

Recordatorios — mis zonas de riesgo

- Soy ansioso: tiendo a avanzar rápido y cerrar incomodidad procesando poco.
- Cierro feedback demasiado rápido ("ya entendí, avancemos") en vez de procesarlo.
- Sub-específico criterios de aceptación y casos borde. Releer y aplicar la taxonomía.
- Esquivo decisiones con "es estándar" / "lo definimos después".
- No releo antes de cerrar. **Trabajar cansado/de noche aumenta el riesgo de aprobar mal.**

Antipatrones a no cometer: vibe coding · spec retroactiva · saltar la adversaria "porque es chica" · modificar spec sin frenar la generación · generar todas las capas de una · aceptar código que "funciona" sin verificar que cumple spec · commitear un fix sin su `bugfix-NNN.md` · cerrar un bug sin test de regresión.

Documento de referencia. Para el detalle operativo completo, ver `workfLow/WORKFLOW.md`.